

Universidad Nacional Mayor de San Marcos
Facultad de Ingeniería de Sistemas e Informática
Escuela Profesional de Ingeniería de Sistemas



TRABAJO GRUPAL:

Gestión de Flujo de Aprobaciones - CAMPUS360

DOCENTE:

Andrade Mogollon, Teodoro Manuel

ASIGNATURA:

DISEÑO DE SISTEMAS DE INFORMACIÓN

SECCIÓN:

INTEGRANTES:

Condori Pari, Arnol
Huaranca Quilca, Gustavo Enrique
Moya Laureano, Jussel Enrique
Ninamango Mucha, Demian Joseph
Quispe Paricahua, Dante Jussie
Ticsihua De La Cruz, Erick Enerson

Lima - Perú
2026

ÍNDICE

1. Introducción	3
1.1. Objetivo del plan de pruebas.....	3
1.2. Alcance	3
2. Entorno y ejecución reproducible	3
2.1. Herramientas	3
2.2. Estructura de carpetas (/tests)	3
2.3. Comando de ejecución de pruebas.....	4
3. Pruebas unitarias (6).....	4
3.1. Casos de prueba unitarios	5
3.2. Evidencia de ejecución	6
3.2.1. Prueba UT-01	6
3.2.2. Prueba UT-02.....	6
3.2.3. Prueba UT-03.....	7
3.2.4. Prueba UT-04.....	7
3.2.5. Prueba UT-05.....	8
3.2.6. Prueba UT-06.....	9
4. Pruebas de caja negra (8)	10
4.1. Endpoints bajo prueba.....	10
4.2. Casos de prueba caja negra	11
4.3. Evidencia.....	14
5. Prueba de caja blanca.....	14
5.1. Unidad bajo prueba:	14
5.2. Cobertura de ramas (if/else).....	15
5.3. Evidencia (ramas/cobertura)	16
6. Resultados y conclusiones.....	17
7. Anexos	17
7.1. Anexo A: Código de pruebas de caja negra.....	17

1. Introducción

El presente plan de pruebas describe cómo se verificará el correcto funcionamiento del backend del módulo G4 – Flujo de Aprobaciones del proyecto Campus360. Se establecen los tipos de pruebas, el entorno de ejecución y la forma de obtener evidencia verificable, priorizando una ejecución reproducible mediante pruebas automatizadas.

1.1. Objetivo del plan de pruebas

- Validar que los endpoints principales del flujo de aprobaciones respondan correctamente ante entradas válidas e inválidas (caja negra).
- Verificar reglas de negocio y validaciones (DTO/Pydantic, lógica de SLA y creación de solicitudes) mediante pruebas unitarias.
- Asegurar cobertura de decisiones internas relevantes mediante una prueba de caja blanca sobre `SolicitudFactory.crear_solicitud()`.
- Garantizar que las pruebas puedan ejecutarse por cualquier evaluador con pasos mínimos y resultados consistentes (reproducibles).

1.2. Alcance

Este plan cubre el backend del módulo de Flujo de Aprobaciones, incluyendo:

- Validaciones de entrada (schemas/DTOs).
- Lógica de negocio relacionada a creación y clasificación de solicitudes (SLA/prioridad).
- Endpoints del módulo: bandeja de pendientes, dictamen, derivación/escala, historial y detalle. No incluye pruebas de interfaz gráfica (frontend), performance, ni seguridad avanzada (autenticación real), salvo validaciones básicas de comportamiento y respuestas HTTP

2. Entorno y ejecución reproducible

2.1. Herramientas

- **Python 3.x** (entorno de ejecución del backend).
- **FastAPI** (framework API).
- **pytest** (framework de pruebas automatizadas).
- **fastapi.testclient (Test Client)** (para pruebas de caja negra automatizadas contra endpoints).

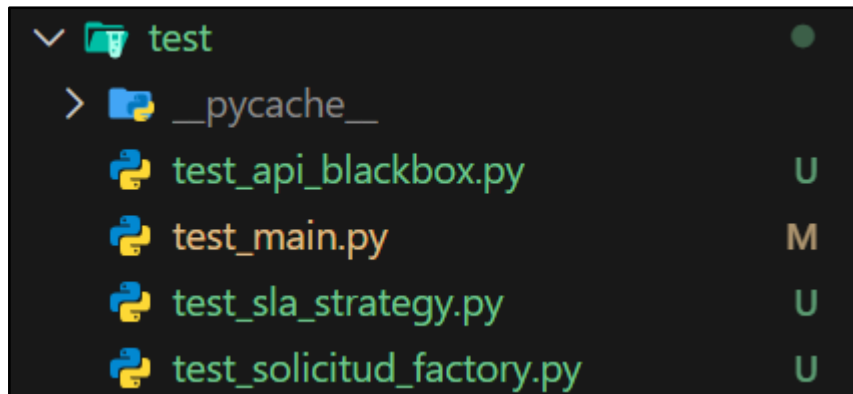
2.2. Estructura de carpetas (/tests)

En `test_api_blackbox.py`, están las 8 pruebas de caja negra

En `test_sla_strategy.py`, están 4 pruebas unitarias

En `test_solicitud_factory.py` están 2 pruebas unitarias, haciendo un total de 6 pruebas unitarias

Observación: La prueba de caja blanca se hizo mediante nodos, eso se observa en el apartado de prueba de caja blanca



2.3. Comando de ejecución de pruebas

Desde la raíz del proyecto, se ejecutan con comandos estándar:

- Ejecutar todas las pruebas:

pytest -q

3. Pruebas unitarias (6)

3.1. Casos de prueba unitarios

ID	Nombre de la prueba	Objetivo	Resultado esperado
UT-01	test_calcular_sla_regular_72_horas	Verificar que el método calcular_sla de la estrategia regular sume exactamente 72 horas a la fecha actual.	Fecha y hora actual + 72 horas (SLA $\approx$ 72h)
UT-02	test_tramite_regular_obtener_prioridad_devuelve_normal	Verificar que la estrategia de trámite regular devuelva la prioridad correcta	"NORMAL"
UT-03	test_tramite_urgente_valida_plazo_exacto_de_24_horas	Verificar que la estrategia de trámite urgente calcule correctamente el tiempo de resolución (SLA) sumando 24 horas a la fecha y hora actual.	Fecha y hora actual + 24 horas (SLA $\approx$ 24h)
UT-04	test_tramite_urgente_verifica_etiqueta_prioridad_alta	Verificar que la estrategia de trámite urgente devuelva la prioridad correcta estipulada por el negocio.	"ALTA"
UT-05	test_crear_solicitud_extemporanea_prioridad_alta_y_sla_24h	Verificar que la Factory identifique trámites "Extemporáneos" y asigne automáticamente la estrategia urgente.	Prioridad "ALTA" y SLA $\approx$ 24h
UT-06	test_factory_crea_tramite_regular_con_prioridad_normal_y_sla_72h	Verificar que la factory cree correctamente una solicitud regular con prioridad y SLA correctos	Prioridad "NORMAL" y SLA $\approx$ 72h

3.2. Evidencia de ejecución

3.2.1. Prueba UT-01

test_sla_strategy.py

```
import pytest
from datetime import datetime, timedelta
from app.services.sla_strategy import TramiteRegularStrategy

def test_calcular_sla_regular_72_horas():
    """
    Prueba unitaria exclusiva para el método calcular_sla
    de la clase TramiteRegularStrategy.
    """
    # Arrange (Organizar)
    estrategia = TramiteRegularStrategy()
    margen_error = 5 # segundos de tolerancia para la ejecución

    # Calculamos el valor esperado localmente para comparar
    tiempo_esperado = datetime.now() + timedelta(hours=72)

    # Act (Actuar)
    resultado = estrategia.calcular_sla()

    # Assert (Afirmar)
    # Verificamos que la diferencia entre el resultado y el esperado sea mínima
    diferencia = abs((resultado - tiempo_esperado).total_seconds())

    assert diferencia < margen_error, f"El SLA calculado ({resultado}) difiere demasiado del esperado ({tiempo_esperado})"
```

Ejecución de Prueba

```
platform win32 -- Python 3.12.1, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\Joseph\Downloads\CAMPUS360_Flujo_Aprobaciones-DSI_2026-backend
plugins: mock-3.15.1
collected 1 item

test\test_sla_strategy.py . [100%]

===== 1 passed in 0.02s =====
```

3.2.2. Prueba UT-02

test_sla_strategy.py

```
from app.services.sla_strategy import TramiteRegularStrategy

def test_tramite_regular_obtener_prioridad_devuelve_normal():
    estrategia = TramiteRegularStrategy()

    prioridad = estrategia.obtener_prioridad()

    assert prioridad == "NORMAL"
```

Ejecución de Prueba

```

===== test session starts =====
platform win32 -- Python 3.13.2, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\user\Downloads\CAMPUS360_Flujo_Aprobaciones-DSI_2026-\backend
plugins: anyio-4.12.1
collected 1 item

test\test_sla_strategy.py . [100%]

===== 1 passed in 0.05s =====

```

3.2.3. Prueba UT-03

test_sla_strategy.py

```

# Prueba para el Punto 3
def test_calcular_sla_devuelve_24_horas_adicionales():
    # 1. Arrange (Preparar)
    estrategia = TramiteUrgenteStrategy()

    # 2. Act (Actuar)
    resultado = estrategia.calcular_sla()

    # 3. Assert (Verificar)
    tiempo_esperado = datetime.now() + timedelta(hours=24)
    diferencia = abs(resultado - tiempo_esperado)

    # Se Verifica que la diferencia sea de menos de 1 segundo
    assert diferencia < timedelta(seconds=1)

```

Ejecución de Prueba

```

DSI_2026-\backend
collected 2 items

test/test_sla_strategy.py::test_calcular_sla_devuelve_24_horas_adicionales PASSED [ 50%]
test/test_sla_strategy.py::test_obtener_prioridad_devuelve_alta PASSED [100%]

===== 2 passed in 0.01s =====

```

3.2.4. Prueba UT-04

test_sla_strategy.py

```

# Prueba para el Punto 4
def test_obtener_prioridad_devuelve_alta():
    # 1. Arrange (Preparar)
    estrategia = TramiteUrgenteStrategy()
    resultado_esperado = "ALTA"

    # 2. Act (Actuar)
    resultado = estrategia.obtener_prioridad()

    # 3. Assert (Verificar)
    assert resultado == resultado_esperado

```

Ejecución de Prueba

```

DSI_2026-\backend
collected 2 items

test/test_sla_strategy.py::test_calcular_sla_devuelve_24_horas_adicionales PASSED [ 50%]
test/test_sla_strategy.py::test_obtener_prioridad_devuelve_alta PASSED [100%]

===== 2 passed in 0.01s =====

```

3.2.5. Prueba UT-05

test_solicitud_factory.py

```

import pytest
from datetime import datetime, timedelta
from app.services.solicitud_factory import SolicitudFactory

def test_crear_solicitud_extemporanea_prioridad_alta_y_sla_24h():
    """
    Verifica que al ingresar 'Matricula Extemporánea', la fábrica asigne:
    - Prioridad: ALTA
    - SLA: Aproximadamente 24 horas (Estrategia Urgente)
    """
    # Arrange (Organizar)
    tipo_tramite = "Matricula Extemporánea"
    solicitante = "Usuario"
    estado_id = 1
    margen_error = 5 # segundos
    esperado_sla = datetime.now() + timedelta(hours=24)

    # Act (Actuar)
    nueva_solicitud = SolicitudFactory.crear_solicitud(tipo_tramite, solicitante, estado_id)

    # Assert (Afirmar)
    # 1. Verificamos que la prioridad sea ALTA
    assert nueva_solicitud.prioridad == "ALTA"

    # 2. Verificamos que el SLA calculado sea el de la estrategia urgente (24h)
    diferencia = abs((nueva_solicitud.slaObjetivo - esperado_sla).total_seconds())
    assert diferencia < margen_error, f"El SLA ({nueva_solicitud.slaObjetivo}) no corresponde a las 24h esperadas"

    # 3. Verificamos que los datos básicos se hayan asignado correctamente
    assert nueva_solicitud.tipoSolicitud == tipo_tramite
    assert nueva_solicitud.solicitante == solicitante

```

Ejecución de Prueba

```
===== test session starts =====
platform win32 -- Python 3.12.1, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\Joseph\Downloads\CAMPUS360_Flujo_Aprobaciones-DSI_2026-\backend
plugins: mock-3.15.1
collected 1 item

test\test_solicitud_factory.py . [100%]

===== 1 passed in 1.41s =====
```

3.2.6. Prueba UT-06

test_solicitud_factory.py

```
from datetime import datetime, timedelta
from app.services.solicitud_factory import SolicitudFactory

def test_factory_crea_tramite_regular_con_prioridad_normal_y_sla_72h():

    tipo_tramite = "Rectificacion de Nota"
    solicitante = "Gustavo"
    estado_inicial_id = 1

    solicitud = SolicitudFactory.crear_solicitud(
        tipo_tramite,
        solicitante,
        estado_inicial_id
    )

    # Validar prioridad
    assert solicitud.prioridad == "NORMAL"

    # Validar SLA (aproximadamente 72h)
    ahora = datetime.now()
    diferencia = solicitud.slaObjetivo - ahora

    # Permitimos margen de error de algunos segundos
    assert timedelta(hours=71, minutes=59) < diferencia < timedelta(hours=72, minutes=1)
```

Ejecución de Prueba

```
===== test session starts =====
platform win32 -- Python 3.13.2, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\user\Downloads\CAMPUS360_Flujo_Aprobaciones-DSI_2026-\backend
plugins: anyio-4.12.1
collected 1 item

test\test_solicitud_factory.py . [100%]

===== 1 passed in 0.54s =====
```

4. Pruebas de caja negra (8)

4.1. Endpoints bajo prueba

Se evaluarán los endpoints del módulo mediante solicitudes HTTP simulando el consumo desde el frontend:

1	GET /api/v1/approvals/pending	Listar bandeja de pendientes
2	GET /api/v1/approvals/{id}/detail	Consultar detalle de una solicitud
3	POST /api/v1/approvals/{id}/verdict	Registrar dictamen
4	POST /api/v1/workflow/{id}/escalate	Evaluación secretarial
5	GET /api/v1/approvals/history	Consultar historial de solicitudes
6	POST /api/v1/approvals/test-seed	Únicamente como apoyo para generar una solicitud de prueba y obtener un id válido, no se cuenta como caja negra

4.2. Casos de prueba caja negra

ID	Endpoint / Método	Entrada (Path / Body)	Precondición	Salida esperada	Criterio de aprobación
CN01	GET /api/v1/approvals/pending	Path: — Body: —	Ninguna	200 OK y respuesta tipo lista (JSON array)	<i>status_code=200</i> y <i>response</i> es <i>list</i>
CN02	GET /api/v1/approvals/{id}/detail	Path: {id}=id_generado Body: —	Existe una solicitud con ese id	200 OK y JSON con <i>idSolicitud</i> , <i>estado_actual</i> , <i>auditoria_decisiones</i> (lista)	<i>status_code=200</i> y <i>id=={id}</i> y <i>auditoria_decisiones</i> es lista
CN03	POST /api/v1/approvals/{id}/verdict	Path: {id}=id_generado Body: {"decision": "APROBADO", "comentario": "Conforme"}	Existe una solicitud con ese id	200 OK y JSON con <i>nuevo_estado="APROBADO"</i>	<i>status_code=200</i> y <i>nuevo_estado=="APROBADO"</i>
CN04	POST /api/v1/approvals/{id}/verdict	Path: {id}=id_generado Body: {"decision": "INVALIDA"}	Existe una solicitud con ese id	400 Bad Request (decisión no permitida)	<i>status_code</i> es 400
CN05	POST /api/v1/approvals/{id}/verdict	Path: {id}=id_generado Body: {"decision": "OBSERVADO", "comentario": "Falta sustento (>=5)."}	Existe una solicitud con ese id	200 OK y JSON con <i>nuevo_estado="OBSERVADO"</i>	<i>status_code=200</i> y <i>nuevo_estado=="OBSERVADO"</i>
CN06	POST /api/v1/workflow/{id}/escalate	Path: {id}=id_generado Body: {"area_destino": "JEFATURA", "checklist_valido": true, "comentario": "Checklist OK"}	Solicitud en estado permitido (PENDIENTE u OBSERVADO)	200 OK y JSON con <i>nuevo_estado="POR_APROBAR"</i>	<i>status_code=200</i> y <i>nuevo_estado=="POR_APROBAR"</i>
CN07	POST /api/v1/workflow/{id}/escalate	Path: {id}=id_generado Body: {"checklist_valido": true} (sin <i>area_destino</i>)	Existe solicitud con ese id	422 Unprocessable Entity (falta campo obligatorio)	<i>status_code</i> es 422

CN08	GET /api/v1/approvals/{id} /detail	Path: <i>{id}=999999999</i> Body: —	Ninguna	404 Not Found	<i>status_code=404</i>
------	---	---	---------	----------------------	------------------------

Para asegurar reproducibilidad, el **id** de los casos CB02–CB07 se obtiene creando una solicitud previa mediante el endpoint **POST /api/v1/approvals/test-seed**, el cual retorna **id_generado**

4.3. Evidencia

Comando de ejecución reproducible

```
pytest -q
```

Evidencia

```
(.venv) PS C:\Users\HP\Desktop\DSI_PROYECTO_2026_0\CAMPUS360_Flujo_Aprobaciones-DSI_2026-\backend> pytest -q
.....
===== warnings summary =====
app\domain\schemas.py:8
  C:\Users\HP\Desktop\DSI_PROYECTO_2026_0\CAMPUS360_Flujo_Aprobaciones-DSI_2026-\backend\app\domain\schemas.p
y:8: PydanticDeprecatedSince20: Support for class-based `config` is deprecated, use ConfigDict instead. Depre
cated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/
2.12/migration/
    class SolicitudDTO(BaseModel):

app\domain\schemas.py:28
  C:\Users\HP\Desktop\DSI_PROYECTO_2026_0\CAMPUS360_Flujo_Aprobaciones-DSI_2026-\backend\app\domain\schemas.p
y:28: PydanticDeprecatedSince20: Support for class-based `config` is deprecated, use ConfigDict instead. Depre
cated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/
2.12/migration/
    @validator('comentario')

app\domain\schemas.py:46
  C:\Users\HP\Desktop\DSI_PROYECTO_2026_0\CAMPUS360_Flujo_Aprobaciones-DSI_2026-\backend\app\domain\schemas.p
y:46: PydanticDeprecatedSince20: Pydantic V1 style `@validator` validators are deprecated. You should migrate
to Pydantic V2 style `@field_validator` validators, see the migration guide for more details. Depreciated in
Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migr
ation/
    @validator('comentario')

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
8 passed, 3 warnings in 2.05s
(.venv) PS C:\Users\HP\Desktop\DSI_PROYECTO_2026_0\CAMPUS360_Flujo_Aprobaciones-DSI_2026-\backend>
```

Captura del terminal con 8 passed, Además, se registran advertencias de compatibilidad por migración Pydantic v1a v2, sin impacto en la ejecución.

Referencia al código

El script de la prueba está en **backend/test/test_api_blackbox.py**

5. Prueba de caja blanca

5.1. Unidad bajo prueba:

```
SolicitudFactory.crear_solicitud()
```

```
@staticmethod
```

```

def crear_solicitud(tipo_tramite: str, solicitante: str, estado_inicial_id: int) -> Solicitud:
    # Decidimos qué estrategia usar basándonos en el nombre del trámite
    estrategia: SlaStrategy
    if "Extemporánea" in tipo_tramite or "Urgente" in tipo_tramite:
        estrategia = TramiteUrgenteStrategy()
    else:
        estrategia = TramiteRegularStrategy()

    # Ensamblamos la entidad (Ya no pasamos el campo historial)
    nueva_solicitud = Solicitud(
        tipoSolicitud=tipo_tramite,
        solicitante=solicitante,
        estado_id=estado_inicial_id,
        slaObjetivo=estrategia.calcular_sla(),
        prioridad=estrategia.obtener_prioridad(),
        adjuntos=[]
    )
    return nueva_solicitud

```

5.2. Cobertura de ramas (if/else)

Código fuente etiquetado

```

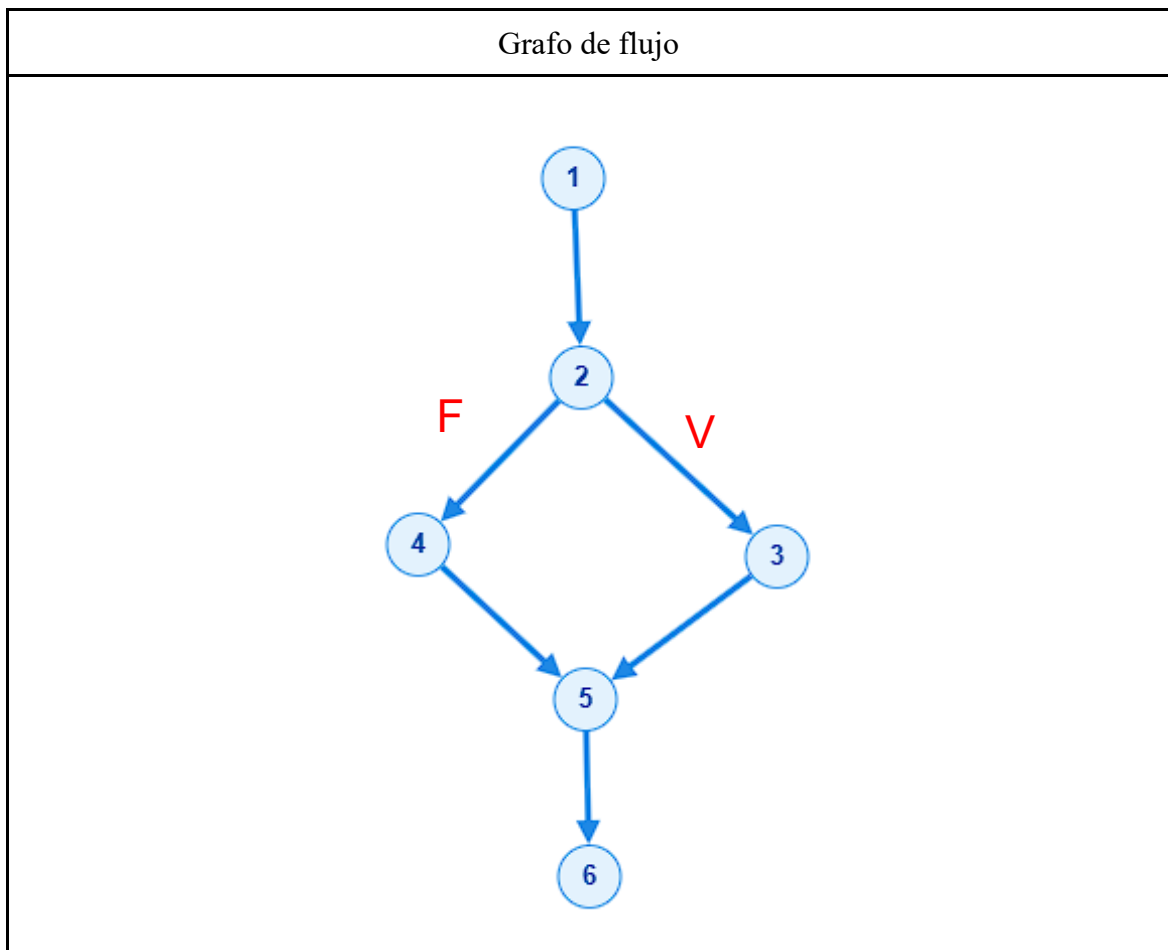
# [ 1 ]
@staticmethod
def crear_solicitud(tipo_tramite: str, solicitante: str, estado_inicial_id: int) ->
Solicitud:

    # Decidimos qué estrategia usar basándonos en el nombre del trámite
    estrategia: SlaStrategy
    # [ 2 ]
    if "Extemporánea" in tipo_tramite or "Urgente" in tipo_tramite:
        # [ 3 ]
        estrategia = TramiteUrgenteStrategy()
    else:
        # [ 4 ]
        estrategia = TramiteRegularStrategy()

    # Ensamblamos la entidad (Ya no pasamos el campo historial)
    # [ 5 ]
    nueva_solicitud = Solicitud(
        tipoSolicitud=tipo_tramite,
        solicitante=solicitante,
        estado_id=estado_inicial_id,
        slaObjetivo=estrategia.calcular_sla(),
        prioridad=estrategia.obtener_prioridad(),
        adjuntos=[]
    )
    # [ 6 ]
    return nueva_solicitud

```

5.3. Evidencia (ramas/cobertura)



5.4. Prueba del camino básico

$$V(G) = A - N + 2$$

$$V(G) = 6 - 6 + 2$$

$$V(G) = 2$$

El conjunto de caminos independientes del grafo será 2.

$$C(1) = 1-2-3-5-6$$

$$C(2) = 1-2-4-5-6$$

Camino	Tipo tramite	Solicitante	Resultado
C(1)	Urgente	Juan	Solicitud creada con TramiteUrgenteStrategy
C(2)	Regular	Ana	Solicitud creada con TramiteRegularStrategy

6. Anexos

7.1. Anexo A: Código de pruebas de caja negra

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)
API_PREFIX = "/api/v1"

def crear_solicitud_prueba(tipo_tramite: str = "Rectificación de
Nota", solicitante: str = "Dante") -> int:
    """
    Crea una solicitud usando el endpoint test-seed para no depender
    de data previa.
    Retorna el id de la solicitud creada.
    """
    payload = {"tipo_tramite": tipo_tramite, "solicitante":
solicitante}
    r = client.post(f"{API_PREFIX}/approvals/test-seed", json=payload)
    assert r.status_code == 200, f"No se pudo crear solicitud:
{r.text}"
    data = r.json()

    if "id_generado" in data:
        return data["id_generado"]

    if "id_solicitud" in data:
        return data["id_solicitud"]
    if "idSolicitud" in data:
        return data["idSolicitud"]
    if "id" in data:
        return data["id"]

    raise AssertionError(f"Respuesta inesperada de test-seed: {data}")
```

```
#PRUEBAS CAJA NEGRA
```

```
def test_cb01_listar_pendientes_responde_200_y_lista():
```

```
    r = client.get(f"{API_PREFIX}/approvals/pending")
```

```
    assert r.status_code == 200
```

```
    assert isinstance(r.json(), list)
```

```
def test_cb02_crear_solicitud_y_ver_que_aparece_en_detalle():
```

```
    solicitud_id = crear_solicitud_prueba("Rectificación de Nota",  
    "Dante")
```

```
    r = client.get(f"{API_PREFIX}/approvals/{solicitud_id}/detail")
```

```
    assert r.status_code == 200
```

```
    data = r.json()
```

```
    assert data["id"] == solicitud_id
```

```
    assert "estado_actual" in data
```

```
    assert "auditoria_decisiones" in data
```

```
    assert isinstance(data["auditoria_decisiones"], list)
```

```
def test_cb03_dictamen_aprobado_actualiza_estado():
```

```
    solicitud_id = crear_solicitud_prueba("Rectificación de Nota",  
    "Dante")
```

```
    r = client.post(
```

```
        f"{API_PREFIX}/approvals/{solicitud_id}/verdict",
```

```
        json={"decision": "APROBADO", "comentario": "Conforme"}  
    )
```

```
    assert r.status_code == 200
```

```
    data = r.json()
```

```
    assert data.get("nuevo_estado") == "APROBADO"
```

```
def test_cb04_dictamen_decision_invalida_debe_fallar():
```

```
    """
```

```
    Caso negativo confiable: el controller bloquea decisiones fuera  
    del set permitido con 400.
```

```
    """
```

```
    solicitud_id = crear_solicitud_prueba("Rectificación de Nota",  
    "Dante")
```

```

    r = client.post(
        f"{API_PREFIX}/approvals/{solicitud_id}/verdict",
        json={"decision": "INVALIDA"}
    )
    assert r.status_code == 400

def test_cb05_dictamen_observado_con_comentario_ok():
    solicitud_id = crear_solicitud_prueba("Rectificación de Nota",
    "Dante")
    r = client.post(
        f"{API_PREFIX}/approvals/{solicitud_id}/verdict",
        json={"decision": "OBSERVADO", "comentario": "Falta sustento
    (>=5)."}
    )
    assert r.status_code == 200
    assert r.json().get("nuevo_estado") == "OBSERVADO"

def test_cb06_escalate_checklist_valido_deriva_a_por_aprobar():
    """
    DerivacionInput exige area_destino (obligatorio) +
    checklist_valido.
    """
    solicitud_id = crear_solicitud_prueba("Rectificación de Nota",
    "Dante")
    r = client.post(
        f"{API_PREFIX}/workflow/{solicitud_id}/escalate",
        json={"area_destino": "JEFATURA", "checklist_valido": True,
    "comentario": "Checklist OK"}
    )
    assert r.status_code == 200
    data = r.json()
    assert data.get("nuevo_estado") == "POR_APROBAR"

def test_cb07_escalate_sin_area_destino_falla_validacion():
    """
    Caso negativo confiable: DerivacionInput exige area_destino
    (obligatorio).
    Si se omite, FastAPI/Pydantic debe retornar 422.
    """

```

```
solicitud_id = crear_solicitud_prueba("Rectificación de Nota",
"Dennis")

r = client.post(
    f"{API_PREFIX}/workflow/{solicitud_id}/escalate",
    json={"checklist_valido": True} # falta area_destino
)
assert r.status_code == 422

def test_cb08_detalle_solicitud_inexistente_devuelve_404():
    r = client.get(f"{API_PREFIX}/approvals/999999999/detail")
    assert r.status_code == 404
```